

Take-Home Assignment

Policy Servicing Mini-Portal

Full Stack Developer · Node.js · React · Oracle Database · Surat (Onsite)

Time to spend	6–8 hours of focused work. Please do not spend more — if you run out of time, write down what you would have done next.
Deadline	2 calendar days from the date you receive this.
Stack	Node.js, React and Oracle Database. No substitutions on the database.
You get	A starter repo with Oracle in Docker, the legacy schema and a dump of legacy data.
You submit	A Git repo with real commit history, plus two filled-in markdown files.
Then	A 60-minute call where you walk us through it and make one small change live.

1 · What we are actually looking for

This is not a puzzle and there are no hidden tricks. It is a small slice of the kind of work you would do on this project: an internal tool for branch staff, over an Oracle database, with business rules that came from a real insurance product.

We care most about three things:

- **Do you read requirements carefully?** Most of the difficulty in this task sits in the edges of the rules, not in the happy path.
- **Do you notice when data does not match its assumptions?** The seed data came out of a system that ran for eleven years. It is messy in the way real data is messy.
- **Does your database layer hold up?** Two clerks clicking the same button at the same moment should not produce two payments.

We are not assessing visual design. A plain, clear screen scores the same as a beautiful one. We would much rather see a correct PL/SQL procedure than a polished dashboard.

2 · The scenario

Branch staff at **Suraksha Life Insurance** (a fictional company, for this exercise) service policies at the counter all day. Today they do it in a tool nobody wants to touch any more. We are rebuilding the smallest useful slice of it. In their words:

“Show me which policies have a premium due, which are in the grace period and which have lapsed — and let me record a payment.”

That is the whole product. Everything below is detail.

3 · Business rules

Please read these twice. Most of the assessment lives in the edges.

Rule	Detail
R1 Premium modes	Yearly, Half-yearly, Quarterly, Monthly.
R2 Grace period	30 days after the due date for Yearly, Half-yearly and Quarterly. 15 days for Monthly.
R3 The last day counts	A payment made on the final day of grace is on time. The business runs in IST (UTC+05:30) . PAYMENTS.PAID_AT is stored in UTC , and your server may well be running in UTC too.
R4 Status is derived	Never stored. Compute it; do not read LEGACY_STATUS . See the table below.
R5 Exact amounts only	A payment must equal the modal premium exactly. Partial payments and overpayments are rejected, with a reason a clerk can read.
R6 One period forward	A payment moves the next due date forward by exactly one mode period. 31 January plus one month is 28 or 29 February — not 2 or 3 March. The same applies at every month-end.
R7 Revival window	A lapsed policy can be revived only within 2 years of its first unpaid due date (FIRST_UNPAID_DUE_DATE). For this exercise, “revive” just means accepting all pending dues in a single transaction. Outside the window the payment is refused.
R8 Never charge twice	Branch staff double-click. Networks retry. Every payment request carries an Idempotency-Key header. The same key must never produce a second payment — and two simultaneous requests for the same policy must not both succeed.

The four statuses (R4)

Status	Meaning
PAID	Nothing is due yet — the next due date is more than 30 days away.
DUE	The next due date falls within the next 30 days and is not yet past.
IN_GRACE	The due date has passed but the policy is still within its grace period.
LAPSED	Grace has ended without payment.

4 · What to build

A · Database (Oracle)

1. **Finish the schema.** `db/init/01_schema.sql` is what the legacy system left us. Decide what it is missing — keys, constraints, data types, indexes — and add it as your own migration under `db/migrations/`. Do not edit the original files; we want to read your migration, not your edits to ours.
2. **Write `RECORD_PAYMENT`** as a PL/SQL procedure. A stub signature is in the schema file; change it if you have a better one, and say why. It must enforce R5–R8 **atomically**, and it must stay correct when two sessions call it for the same policy at the same instant.
3. **Write the derived-status query (R4)** as a view or a query, with filtering and pagination done in the database. `V_POLICY_STATUS` is a stub.

Some of your migrations will fail the first time you run them. That is expected, and it is informative — the legacy data does not satisfy the constraints a sane schema would have. Deal with each conflict and write down what you decided and why. We are more interested in the reasoning than in a clean run.

For every index you add, say in `NOTES.md` which query it serves. An index with no query behind it counts against you.

B · Backend (Node.js + node-oracledb)

Endpoint	What it does
<code>GET /policies</code>	Query params: <code>status</code> , <code>search</code> , <code>page</code> , <code>pageSize</code> . Filter by derived status; search policy number or customer name, case-insensitive; paginate in the database; return the total count alongside the rows.
<code>GET /policies/:id</code>	Policy, customer, derived status and payment history, newest first.
<code>POST /policies/:id/payments</code>	Records a payment through your procedure. Honours the Idempotency-Key header. Body: <code>{ "amount": 12500.00, "channel": "BRANCH" }</code> .

Also expected: bind variables everywhere, a connection pool used correctly including on the error path, and HTTP status codes that mean something. A rejected business rule is not a 500. Return a machine-readable error code alongside the human message.

C · Frontend (React)

- **Policy list** — status filter, search, server-side pagination. Status legible at a glance, and not conveyed by colour alone.
- **Policy detail** — details, payment history, and a **Record Payment** form.
- The form must be impossible to submit twice: double-click, Enter key, slow network. It must show the API's business-rule errors in language a branch clerk would understand.
- Loading, empty and error states everywhere. Amounts formatted as INR.

Routing, state management and styling are your call. A component library is fine. Plain CSS is fine.

D · Code review

`legacy/paymentService.js` is running in production today. **Do not fix it.** Review it. In `REVIEW.md`, list what you find: what is wrong, **why it matters in production**, and how you would fix it. The “why it matters” is the part we read most closely — naming a problem is easy, explaining its consequence is the skill.

E · NOTES.md

Fill in the template. In particular:

- How to run your work, from zero, including your migrations.
- The decisions you made and what you traded away.
- **Anything you found odd in the data**, and what you did about it.
- What you did not get to, and what you would do with another day.
- Your AI usage — see section 6.

5 · Getting set up

Full instructions are in [README.md](#) in the repo. The short version:

```
docker compose up -d
docker compose logs -f oracle      # wait for "DATABASE IS READY TO USE!"

# terminal 2
cd backend && cp .env.example .env && npm install && npm run dev

# terminal 3
cd frontend && npm install && npm run dev
```

The first database start takes **3–6 minutes** and needs roughly 4 GB free for Docker. The schema and seed scripts run automatically on that first start. `node-oracledb` 6 runs in Thin mode, so you do not need Oracle Instant Client installed.

Connection details: `localhost:1521/FREEPDB1`, user `suraksha`, password `suraksha`.

What is in the repo

Path	What it is
<code>db/init/01_schema.sql</code>	The legacy schema. Read-only — do not edit.
<code>db/init/02_seed.sql</code>	The legacy data dump: 155 customers, 194 policies, 472 payments. Read-only.
<code>db/migrations/</code>	Your migrations go here.
<code>backend/</code>	Express + node-oracledb skeleton with the routes stubbed.
<code>frontend/</code>	Vite + React skeleton.
<code>legacy/paymentService.js</code>	The file to review. Do not fix it.
<code>NOTES.md</code> , <code>REVIEW.md</code>	Templates for you to fill in.

Data dictionary

Three tables. The column names below are the ones that matter.

Table	Columns worth knowing
CUSTOMERS	<code>CUSTOMER_ID</code> , <code>FULL_NAME</code> , <code>PAN</code> , <code>EMAIL</code> , <code>MOBILE</code> , <code>CITY</code>
POLICIES	<code>POLICY_ID</code> , <code>POLICY_NO</code> , <code>CUSTOMER_ID</code> , <code>PLAN_NAME</code> , <code>PREMIUM_MODE</code> , <code>PREMIUM_AMOUNT</code> , <code>SUM_ASSURED</code> , <code>COMMENCEMENT_DATE</code> , <code>NEXT_DUE_DATE</code> , <code>FIRST_UNPAID_DUE_DATE</code> , <code>LEGACY_STATUS</code>
PAYMENTS	<code>PAYMENT_ID</code> , <code>POLICY_ID</code> , <code>AMOUNT</code> , <code>PAID_AT (UTC)</code> , <code>IDEMPOTENCY_KEY</code> , <code>COVERS_DUE_DATE</code> , <code>CHANNEL</code>

`NEXT_DUE_DATE` is the date the next unpaid instalment is due. `FIRST_UNPAID_DUE_DATE` is the due date of the oldest instalment still unpaid — it is what the two-year revival window is measured from — and it is NULL when nothing is outstanding.

6 · About AI tools

Use them. We do, every day. There is one condition and we take it seriously:

| *You must be able to explain and modify every line you submit.*

Tell us in `NOTES.md` which tools you used and for what.

7 · How we assess it

Out of 100. We are showing you the weighting because we would rather you spent your time where it counts.

Area	Weight	What a strong answer looks like
Oracle and SQL	25	Correct constraints, indexes with a reason behind each one, an atomic procedure, concurrency handled deliberately, bind variables, correct month arithmetic
Node backend	20	Pool used correctly, clean error mapping, idempotency that actually works, no leaked connections
React frontend	20	Server-side pagination, double-submit prevented, real loading and error states, status readable at a glance
Business rules and edges	15	Grace boundaries, IST vs UTC, month-end, revival window
Code review	10	Finds the problems and explains their consequences, not just their names
Git history and NOTES	10	Incremental commits, honest trade-offs, data issues documented

8 · Submitting

- A **Git repo** — a GitHub or GitLab link, or a zip that includes the `.git` folder.
- **Keep your real commit history. Please do not squash it into one commit.** We look at how the work was built up, and a single “initial commit” tells us nothing. Messy, honest history is better than a tidy invented one.
- Everything must run from a clean clone with `docker compose up` plus whatever you document.
- `NOTES.md` and `REVIEW.md` completed.

9 · If you get stuck

A blocked environment says nothing about you as a developer. If Docker or Oracle will not cooperate after about thirty minutes, email us — we would much rather unblock you than have you lose an evening to it.

If something in the rules is ambiguous, **make a decision, write it down in `NOTES.md`, and carry on.** Noticing the ambiguity is worth more to us than guessing what we meant. Real requirements arrive exactly like this.

Thank you for the time you are putting into this. We know a take-home is a real cost, and we read every submission properly.